

Spark Repartition vs Coalesce

Understanding Shuffle, DAG and Partition Movement

A visual guide for Spark developers

Spark Execution Starts With a DAG

Driver Builds DAG Read Data ↓ Transformations ↓ Shuffle Boundary ↓ Next Stage Tasks

- Driver builds the DAG
- Spark splits jobs into stages
- Shuffle creates new stage boundaries

What Happens During repartition()

Executor 1 Executor 2 Executor 3 P1 P2 P3 ↓↓↓ SHUFFLE NETWORK ↓↓↓
P4 P5 P6 (Evenly redistributed)

- Triggers a full shuffle
- Data moves across executors
- Creates balanced partitions

What Happens During coalesce()

Executor Partition 1 Partition 2 Partition 3 ↓ Merge Larger Partition

- Avoids full shuffle
- Partitions merged locally
- Less network movement

Driver vs Executors

Driver	Executors
Build DAG	Run tasks
Plan stages	Process partitions
Decide shuffle	Move shuffled data

Spark Performance Insight

- `repartition()` = shuffle + balanced data
- `coalesce()` = minimal shuffle
- Driver plans execution
- Executors perform computation

Understanding partition movement can dramatically improve pipeline performance.